

Estimando a Conformidade de SLAs em um Serviço de Vídeo sob Demanda via Regressão e Classificação

PGC308A: Tópicos Especiais em Sistemas de Computação 2 – Internet do Futuro

Anna Letycia Fernandes Reis – 12612CCP008

João Pedro Ramires Esteves – 12522CCP018

Universidade Federal de Uberlândia (UFU)

Programa de Pós-Graduação em Ciência da Computação

22 de junho de 2026

Resumo

Este relatório apresenta a análise experimental do problema de estimação de conformidade com Acordos de Nível de Serviço (SLAs) em um serviço de *Video on Demand* (VoD). O objetivo é estimar a taxa de quadros observada no cliente a partir de estatísticas coletadas no servidor e, em seguida, classificar cada observação como conforme ou violadora do SLA definido (taxa mínima de 18 quadros - *frames* - por segundo). Foram executadas três etapas: i) exploração dos dados; ii) regressão linear para estimação da métrica de serviço; e iii) regressão logística para classificação de conformidade. Os resultados mostram que a regressão linear atingiu NMAE (*Normalized Mean Absolute Error*) de 10,39%, abaixo do limite de 15% estabelecido no enunciado, enquanto o melhor classificador logístico obteve ERR (*Classification Error*) de 11,11%, também satisfazendo o critério de acurácia. A análise dos gráficos e das tabelas indica que os erros se concentram principalmente nas regiões de transição entre regimes de operação do servidor, especialmente nas proximidades do limiar de SLA.

Sumário

1	Introdução	2
2	Descrição do Conjunto de Dados	2
3	Task I - Exploração dos Dados	3
3.1	Estatísticas Descritivas	3
3.2	Consultas sobre o Conjunto de Dados	3
3.3	Análise Gráfica	4
3.3.1	Série Temporal de CPU Idle e Memória Utilizada	4
3.3.2	Estimativas de Densidade (KDE)	4
4	Task II - Estimação da Métrica de Serviço a partir das Estatísticas de Dispositivo	5
4.1	Formulação e Divisão Treino/Teste	5
4.2	Avaliação da Acurácia de Estimação	5
4.2.1	Coefficientes do Modelo	5
4.2.2	Acurácia do Modelo (NMAE)	6
4.2.3	Série Temporal: Estimativas <i>vs.</i> Medições	6
4.2.4	KDE do <i>Frame Rate</i> no Conjunto de Teste	7
4.2.5	KDE dos Erros Absolutos de Predição	8
4.2.6	Discussão da Acurácia	8
4.3	Relação entre Tamanho do Treino, Acurácia e Tempo de Treinamento	8
4.3.1	Coefficientes dos Modelos	8
4.3.2	Tempo de Treinamento e NMAE	9
5	Task III - Estimação de Conformidade e Violação do SLA a partir das Estatísticas de Dispositivo	10
5.1	Transformação do Problema em Classificação Binária	10
5.2	Impacto do Tamanho do Conjunto de Treinamento no Desempenho do Modelo	11
5.2.1	Coefficientes dos Classificadores	11
5.2.2	Tempo de Treinamento e Erro de Classificação (ERR)	11
5.2.3	Número de Iterações	12
5.3	Análise do Melhor Classificador	13
5.4	Relação entre Tamanho do Treino e Acurácia dos Classificadores	14
5.5	Relação entre Tamanho do Treino e Tempo de Treinamento	14
6	Conclusão	15
7	Referências	16

1 Introdução

Serviços de telecomunicações e de Internet dependem, cada vez mais, de infraestruturas de nuvem distribuída, computação de borda, CDNs e arquiteturas de rede programáveis. Nesse contexto, garantir a qualidade de serviço (QoS) percebida pelo cliente exige mecanismos de observabilidade capazes de relacionar métricas internas da infraestrutura a indicadores de experiência do usuário. Um *Service Level Agreement* (SLA) formaliza esse compromisso, definindo um limiar mínimo de desempenho que o provedor deve sustentar.

Este trabalho investiga a conformidade de um serviço de vídeo sob demanda (VoD) a um SLA definido em termos da taxa de quadros (*frame rate*) observada no terminal do cliente. A abordagem proposta é orientada a dados: a partir de estatísticas do sistema operacional coletadas no servidor (CPU, memória, processos, rede etc.), constrói-se um modelo de aprendizado de máquina $M : X \rightarrow \hat{Y}$ capaz de estimar a métrica de serviço Y sem a necessidade de instrumentar o lado do cliente. O problema é tratado tanto como regressão (estimação do valor contínuo da taxa de quadros) quanto como classificação binária (conformidade ou violação ao SLA), utilizando, respectivamente, regressão linear e regressão logística como modelos-base interpretáveis.

O restante do relatório está organizado da seguinte forma: a Seção 2 descreve o conjunto de dados utilizado; a Seção 3 apresenta a exploração estatística dos dados (Task I); a Seção 4 discute a estimação da taxa de quadros via regressão linear (Task II); a Seção 5 apresenta o classificador de conformidade ao SLA via regressão logística (Task III); e a Seção 6 sintetiza as conclusões do trabalho.

2 Descrição do Conjunto de Dados

O conjunto de dados contém 3600 observações coletadas em intervalos de um segundo ao longo de uma hora de operação de um servidor de VoD. Cada observação relaciona nove estatísticas de dispositivo do servidor (variáveis explicativas X) à taxa de quadros exibida no cliente naquele instante (variável resposta Y). Por clareza de notação, as features originais foram renomeadas conforme a Tabela 1.

Tabela 1: Dicionário de variáveis do conjunto de dados.

Variável original	Nome utilizado	Descrição	Unidade
<code>all_..idle</code>	<code>cpu.idle</code>	Percentual de tempo ocioso da CPU	%
<code>X..memused</code>	<code>mem.used</code>	Percentual de memória utilizada	%
<code>proc.s</code>	<code>proc.creation.rate</code>	Taxa de criação de processos	processos/s
<code>cswch.s</code>	<code>ctx.switch.rate</code>	Taxa de troca de contexto	trocas/s
<code>file.nr</code>	<code>file.handles</code>	Número de <i>file handles</i> em uso	contagem
<code>sum_intr.s</code>	<code>interrupt.rate</code>	Taxa de interrupções	interrupções/s
<code>ldavg.1</code>	<code>load.avg.1min</code>	<i>Load average</i> no último minuto	adimensional
<code>tcpsck</code>	<code>tcp.sockets</code>	Número de sockets TCP em uso	contagem
<code>pgfree.s</code>	<code>pg.free.rate</code>	Taxa de liberação de páginas de memória	páginas/s
<code>DispFrames</code>	<code>frame.rate</code>	Taxa de quadros exibidos	frames/s

O SLA do serviço é definido em termos do *frame rate*: diz-se que o serviço **conforma** ao SLA em um instante se $Y \geq 18$ fps, e que **viola** o SLA caso contrário. Esse limiar é utilizado tanto na Task II, como referência visual de qualidade, quanto na Task III, como definição do rótulo binário a ser classificado.

3 Task I - Exploração dos Dados

3.1 Estatísticas Descritivas

A primeira etapa consistiu em calcular, para cada variável de X e para a variável Y , média, desvio padrão amostral, mínimo, percentil 25, percentil 90 e máximo. Os cálculos foram realizados de duas formas, uma com NumPy e outra com Pandas, e os resultados foram comparados por meio de verificação cruzada, confirmando que ambos os métodos produziram resultados idênticos. A Tabela 2 resume os valores obtidos.

Tabela 2: Estatísticas descritivas das variáveis do conjunto de dados.

Variável	Média	Desvio	Mín.	P25	P90	Máx.
cpu.idle (%)	9.06	16.122	0.00	0.00	38.62	69.54
mem.used (%)	89.13	8.18	73.03	82.96	96.77	97.84
proc.creation.rate	7.68	8.53	0.00	0.00	20.00	48.00
ctx.switch.rate	54,045.87	19,497.81	11,398.00	31,302.00	72,135.10	83,880.00
file.handles	2,656.33	196.11	2,304.00	2,496.00	2,880.00	2,976.00
interrupt.rate	19,978.04	4,797.27	10,393.00	16,678.00	28,228.40	35,536.00
load.avg.1min	75.87	43.86	11.13	28.20	127.99	147.47
tcp.sockets	48.99	15.87	21.00	34.00	71.00	87.00
pg.free.rate	72,872.15	19,504.32	15,928.00	61,601.75	97,532.50	145,874.00
frame.rate (Y)	18.81	5.21	0.00	13.39	24.61	30.39

A Tabela 2 mostra que o servidor opera, na maior parte do tempo, com alta utilização de recursos. O uso médio de memória é de 89,14%, com o primeiro quartil em 82,97% e o percentil 90 alcançando 96,77%. Já a ociosidade média da CPU é de apenas 9,07%, sendo que pelo menos 25% das observações registram 0% de CPU livre. Esse comportamento indica um ambiente frequentemente próximo da saturação. Além disso, a taxa média de quadros é de 18,82 fps, valor muito próximo ao limite de SLA de 18 fps, sugerindo que pequenas oscilações na carga podem ser suficientes para levar o sistema a situações de conformidade ou violação. Os desvios padrão elevados de `load.avg.1min` e `ctx.switch.rate` também indicam uma variação significativa na carga ao longo do tempo, aspecto que será explorado nas próximas análises.

3.2 Consultas sobre o Conjunto de Dados

Três consultas específicas foram realizadas sobre os dados, com o objetivo de quantificar condições de operação sob alta carga. Os resultados, calculados de forma equivalente via filtragem booleana em NumPy e em pandas, são apresentados na Tabela 3.

Tabela 3: Resultados das consultas sobre o conjunto de dados (3,600 observações).

Consulta	Resultado
(a) Observações com <code>mem.used > 80%</code>	2,875 (79,86%)
(b) Média de <code>tcp.sockets</code> quando <code>interrupt.rate > 18,000/s</code>	46,35 sockets
(c) Mínimo de <code>mem.used</code> quando <code>cpu.idle < 20%</code>	73,03%

O item (a) confirma que a alta ocupação de memória é uma condição frequente do servidor: 2875 das 3600 observações, superando os 80% em quase 80% do tempo monitorado. Já o item (b) mostra que, mesmo sob tráfego intenso com mais de 18.000 interrupções por segundo, o número médio de *sockets* TCP ativos (46,35) permanece muito próximo da média global (48,99), indicando que a quantidade de conexões TCP não varia diretamente com a taxa de interrupções. Por fim, o item (c) reforça esse cenário de sobrecarga ao evidenciar que, mesmo nos momentos de menor ociosidade da CPU (abaixo de 20%), o uso de memória nunca fica abaixo de 73,03%, que corresponde ao menor valor

observado para essa métrica em todo o conjunto de dados. Em conjunto, os resultados confirmam a caracterização de um ambiente que opera sob carga elevada durante a maior parte do tempo.

3.3 Análise Gráfica

3.3.1 Série Temporal de CPU Idle e Memória Utilizada

A Figura 1 apresenta as séries temporais de `cpu.idle` e `mem.used` ao longo dos 3600 segundos de monitoramento, plotadas no mesmo eixo (ambas em %).

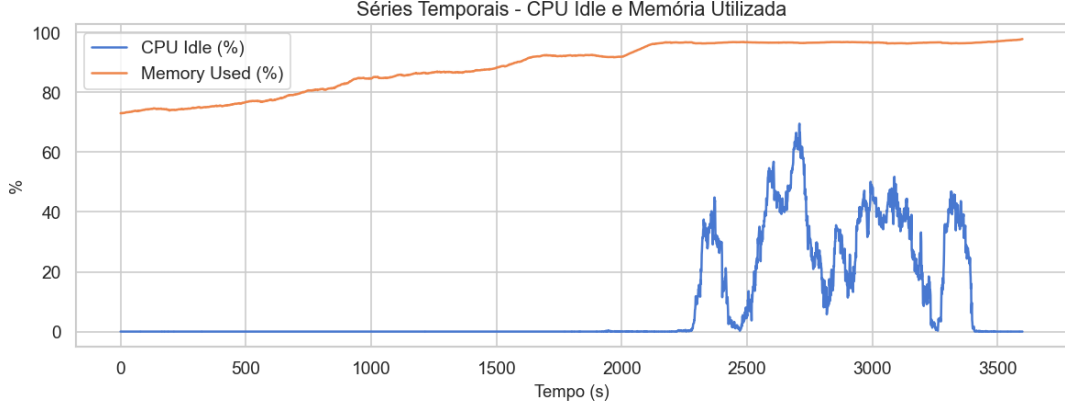


Figura 1: Séries temporais de CPU ociosa e memória utilizada.

A série temporal mostra que CPU e memória apresentam comportamentos distintos ao longo do período analisado. Na primeira parte da coleta (aproximadamente do segundo 0 ao 2200), a CPU permanece praticamente saturada (`cpu.idle` $\approx 0\%$) enquanto a memória cresce de forma contínua de 73% para perto de 97%. A partir do segundo 2200, a CPU passa a apresentar picos intermitentes de ociosidade de até 70%, enquanto a memória permanece estável em um patamar elevado, entre 96% e 98%. Esses resultados sugerem que o sistema opera sob carga elevada durante grande parte do tempo, com a memória mantendo níveis altos de utilização mesmo nos momentos em que a CPU apresenta algum alívio. Esse comportamento indica mudanças no regime de operação ao longo da coleta e ajuda a explicar parte da variabilidade observada nas demais métricas.

3.3.2 Estimativas de Densidade (KDE)

A Figura 2 apresenta as estimativas de densidade de *kernel* (KDE) para `cpu.idle` e `mem.used`, calculadas com *kernel* Gaussiano e largura de banda definida pela regra de Silverman ($h^* = 1,06 \hat{\sigma} n^{-1/5}$). As curvas obtidas pela implementação manual e pela função `KernelDensity` do `scikit-learn` coincidem exatamente, validando a implementação.

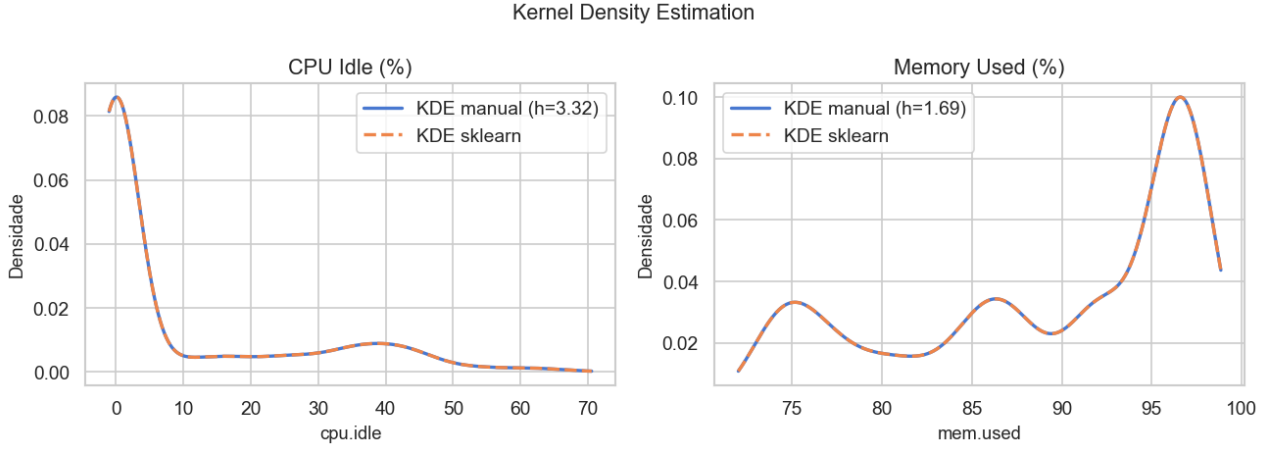


Figura 2: KDE de CPU ociosa (esquerda) e memória utilizada (direita).

A análise do gráfico de densidade revela que a KDE de `cpu.idle` concentra massa próxima de zero, indicando que a CPU permaneceu com baixa ociosidade durante grande parte do período analisado. Embora existam observações com níveis mais elevados de ociosidade, elas ocorrem com menor frequência. Já a distribuição de `mem.used` concentra-se em valores altos e apresenta mais de uma região de maior densidade, com destaque para a faixa entre 96% e 97%. As duas curvas mostram que a memória permanece constantemente demandada, enquanto a CPU responde de forma mais dinâmica às variações da carga de trabalho.

4 Task II - Estimação da Métrica de Serviço a partir das Estatísticas de Dispositivo

4.1 Formulação e Divisão Treino/Teste

Na Task II, o objetivo foi treinar um modelo de regressão linear múltipla $M : X \rightarrow \hat{Y}$, em que X contém as nove estatísticas do dispositivo e Y corresponde ao `frame.rate`. O modelo ajustado tem a forma:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_9 x_9. \quad (1)$$

Seguindo a técnica de *validation-set*, o conjunto de 3600 observações foi particionado aleatoriamente em 70% para treino (2520 observações) e 30% para teste (1080 observações), utilizando `train_test_split` do `scikit-learn` com semente fixa (`random_state=42`) para garantir reprodutibilidade:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=SEED
)
```

O modelo foi ajustado por Mínimos Quadrados Ordinários (OLS), utilizando essas nove estatísticas como variáveis explicativas.

4.2 Avaliação da Acurácia de Estimação

4.2.1 Coeficientes do Modelo

Inicialmente, foi treinado um modelo de regressão linear com o conjunto de treino completo (2520 observações). O intercepto obtido foi $\theta_0 = 49,69$. A Tabela 4 apresenta os coeficientes ordenados por magnitude absoluta.

Tabela 4: Coeficientes do modelo linear treinado com 2520 observações.

Variável	Coeficiente	Interpretação
<code>cpu.idle</code>	-0,0921	Coeficiente de maior magnitude; o sinal negativo é contraintuitivo e sugere colinearidade com outras variáveis de carga.
<code>mem.used</code>	-0,0868	Maior uso de memória está associado à redução do <i>frame rate</i> .
<code>tcp.sockets</code>	-0,0653	Mais conexões simultâneas estão associadas a maior sobrecarga.
<code>load.avg.1min</code>	-0,0606	Maior carga média reduz a capacidade do servidor de manter a taxa de quadros.
<code>proc.creation.rate</code>	-0,0120	Impacto menor, porém ainda negativo.
<code>file.handles</code>	-0,0031	Impacto marginal.
<code>ctx.switch.rate</code>	-0,0001	Efeito praticamente nulo na escala observada.
<code>interrupt.rate</code>	0,0000	Efeito negligenciável.
<code>pg.free.rate</code>	-0,0000	Efeito negligenciável.

A interpretação dos coeficientes indica que variáveis relacionadas à carga do sistema estão, em geral, associadas à redução da taxa de *frame rate* estimada. O intercepto ($\theta_0 = 49,69$) representa o valor teórico do *frame rate* na ausência de carga, servindo como um limite superior do modelo dentro da formulação linear. De forma consistente com a Tabela 4, todos os coeficientes são negativos ou próximos de zero, sugerindo que o aumento no uso de recursos tende a degradar o desempenho. As variáveis `cpu.idle`, `mem.used`, `tcp.sockets` e `load.avg.1min` concentram os efeitos de maior magnitude, enquanto métricas como `ctx.switch.rate`, `interrupt.rate` e `pg.free.rate` apresentam impacto praticamente nulo na predição.

O coeficiente negativo de `cpu.idle` chama atenção, pois isoladamente seria esperado que mais CPU ociosa resultasse em uma taxa de *frame rate* maior. Essa inversão é um forte indício de colinearidade entre as *features*. Como as variáveis de memória, carga média e *sockets* TCP já absorvem a maior parte da informação sobre a carga do servidor, o coeficiente de `cpu.idle` passa a refletir apenas um efeito residual condicional, e não o seu impacto isolado. Isso indica que os coeficientes devem ser interpretados em conjunto dentro do modelo, e não como efeitos causais independentes.

4.2.2 Acurácia do Modelo (NMAE)

O erro do modelo no conjunto de teste foi avaliado pelo Erro Absoluto Médio Normalizado (NMAE):

$$\text{NMAE} = \frac{1}{\bar{y}} \left(\frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \right) \quad (2)$$

Os resultados obtidos foram: $\text{MAE} = 1,9371$ fps, $\bar{y} = 18,65$ fps (média da variável resposta no conjunto de teste) e, portanto, $\text{NMAE} = 0,1039$ (10,39%). Como $10,39\% < 15\%$, o critério de acurácia estabelecido pelo enunciado é satisfeito, classificando o modelo como **acurado**.

4.2.3 Série Temporal: Estimativas *vs.* Medições

A Figura 3 compara, no conjunto de teste, o *frame rate* medido e a estimativa do modelo, apresentando as médias móveis de 30 segundos no painel superior para suavizar o ruído ponto-a-ponto e os resíduos individuais no painel inferior.

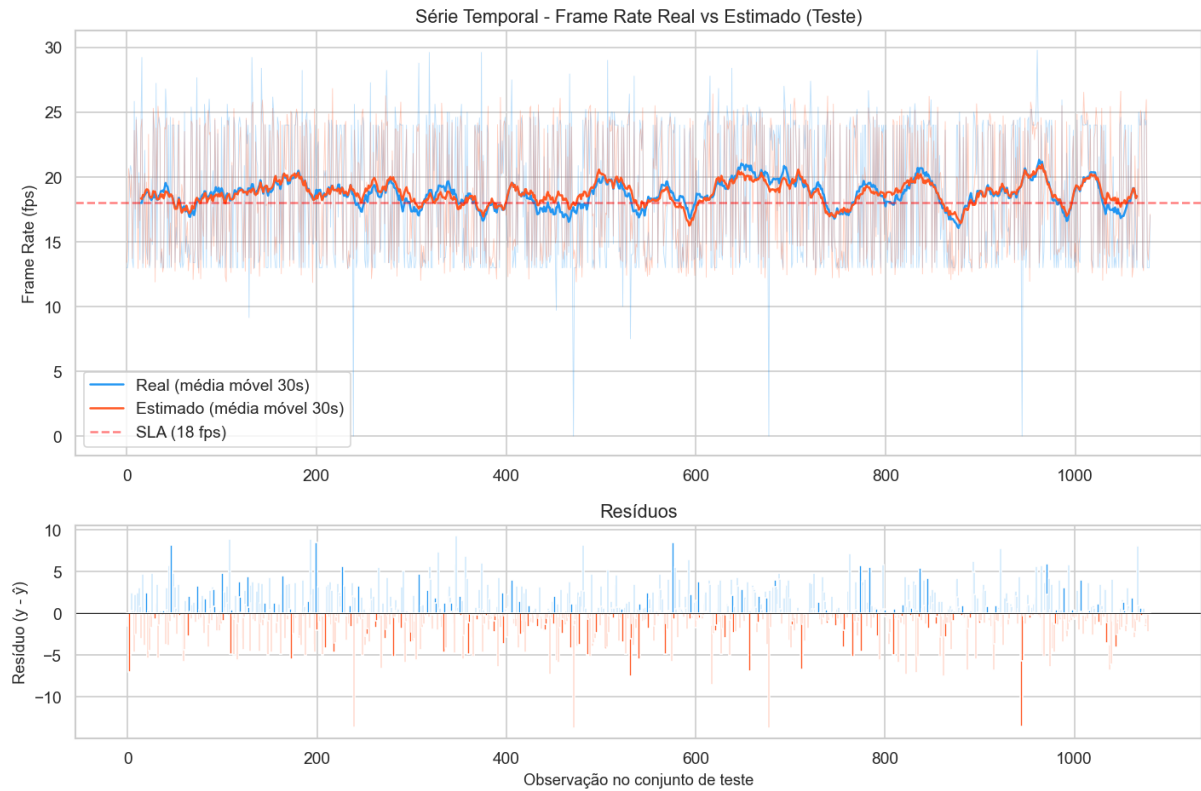


Figura 3: Série temporal do *frame rate* real e estimado no teste, com resíduos.

A curva estimada acompanha bem o comportamento real ao longo de todo o conjunto de teste, capturando com precisão as oscilações de médio prazo em torno do limiar de SLA, representado pela linha pontilhada em 18 fps. As diferenças entre os valores reais e estimados permanecem pequenas na maior parte do tempo, embora ocorram picos isolados de erro de aproximadamente ± 10 fps em alguns pontos específicos. Esses casos coincidem com mudanças rápidas no comportamento do sistema, nas quais a regressão linear tende a suavizar as predições e não consegue acompanhar a velocidade da mudança do valor real.

4.2.4 KDE do *Frame Rate* no Conjunto de Teste

A Figura 4 apresenta a KDE do *frame rate* real no conjunto de teste, com indicação do limiar de SLA de 18 fps.

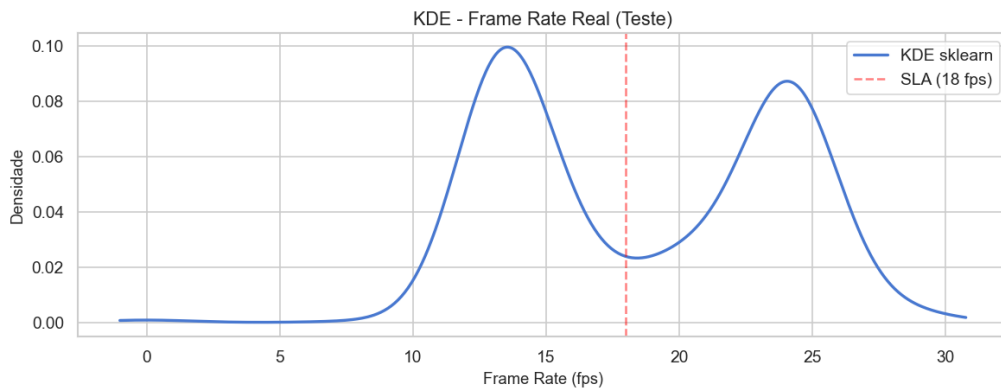


Figura 4: KDE do *frame rate* real no conjunto de teste.

A distribuição do *frame rate* no conjunto de teste é claramente **bimodal**, com dois picos bem definidos próximos de 13 fps e de 24 fps, e um vale exatamente sobre o limiar de SLA (18 fps). Isso indica que o servidor opera predominantemente em dois estados distintos: um regime degradado, abaixo

do SLA, e um regime saudável, acima do SLA. O pico em torno de 13 fps é levemente mais alto, sugerindo que o regime degradado é o mais frequente no conjunto de teste analisado. Esse padrão ajuda a explicar parte dos erros do modelo linear, já que a mudança entre os dois regimes ocorre de forma relativamente abrupta. Tal comportamento é consistente com sistemas saturados, nos quais a degradação do desempenho tende a ocorrer de maneira não linear quando determinados recursos se aproximam de seus limites de operação.

4.2.5 KDE dos Erros Absolutos de Predição

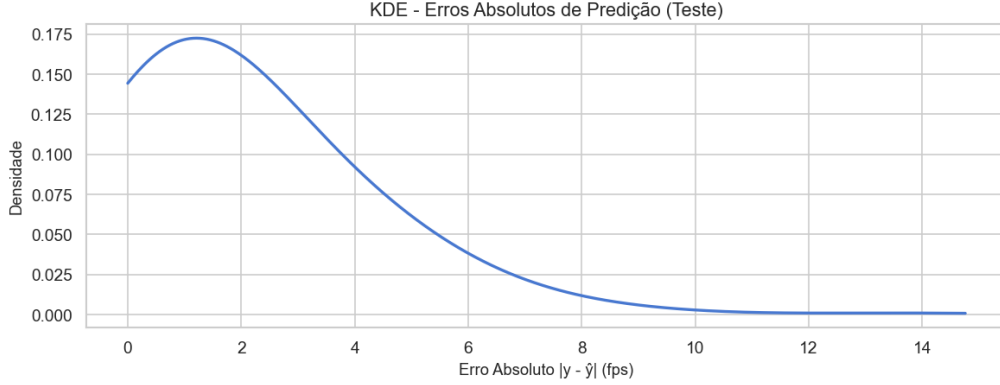


Figura 5: KDE dos erros absolutos de predição no conjunto de teste.

A Figura 5 mostra que os erros absolutos concentram-se principalmente entre 0 e 4 fps, com pico próximo de 1 fps. A distribuição, entretanto, apresenta uma assimetria à direita e uma cauda que alcança aproximadamente 14 fps, indicando a ocorrência de alguns erros maiores. Esses casos podem estar relacionados à distribuição bimodal observada na Figura 4, na qual o sistema alterna entre dois regimes distintos de operação, sugerindo que os maiores erros tendem a ocorrer nos momentos de transição entre esses regimes.

4.2.6 Discussão da Acurácia

O modelo linear atingiu NMAE de 10,39%, valor inferior ao limite de 15% estabelecido no projeto, indicando boa capacidade de predição do *frame rate*. Entretanto, essa métrica representa um erro médio e não evidencia diferenças de desempenho ao longo de toda a série temporal. As análises apresentadas anteriormente mostram que o modelo reproduz adequadamente os dois regimes predominantes de operação, mas encontra maior dificuldade nos momentos de transição entre eles. Esse comportamento é consistente com a distribuição bimodal observada para o *frame rate* e ajuda a explicar a presença de erros mais elevados em uma pequena parcela das observações. Além disso, os coeficientes obtidos sugerem que variáveis relacionadas à utilização de CPU, memória, carga do sistema e número de conexões TCP exercem maior influência sobre as predições. Como a regressão linear assume relações aditivas entre as variáveis, parte da dinâmica do sistema pode não ser completamente capturada, especialmente em situações de mudança abrupta de regime.

4.3 Relação entre Tamanho do Treino, Acurácia e Tempo de Treinamento

Para investigar o efeito do tamanho do conjunto de treino, cinco subconjuntos de tamanhos $n \in \{50, 500, 1000, 1500, 2520\}$ foram amostrados aleatoriamente (com semente fixa) a partir do conjunto de treino original de 2520 observações, gerando cinco modelos $M_1, \dots, M_5$.

4.3.1 Coeficientes dos Modelos

A Tabela 5 apresenta os coeficientes de cada um dos cinco modelos.

Tabela 5: Coeficientes dos modelos lineares para diferentes tamanhos de treinamento.

Parâmetro	$n = 50$	$n = 500$	$n = 1000$	$n = 1500$	$n = 2520$
Intercepto	36.8386	45.0330	52.2503	49.7976	49.6878
cpu.idle	-0.0383	-0.1004	-0.0789	-0.0884	-0.0921
mem.used	0.0263	-0.0611	-0.1006	-0.0790	-0.0868
proc.creation.rate	-0.0497	-0.0336	-0.0183	-0.0114	-0.0120
ctx.switch.rate	-0.0000	-0.0001	-0.0001	-0.0001	-0.0001
file.handles	-0.0066	-0.0025	-0.0044	-0.0037	-0.0031
interrupt.rate	0.0001	0.0000	0.0001	0.0000	0.0000
load.avg.1min	-0.0444	-0.0711	-0.0593	-0.0565	-0.0606
tcp.sockets	-0.0441	-0.0392	-0.0609	-0.0698	-0.0653
pg.free.rate	0.0000	-0.0000	-0.0000	-0.0000	-0.0000

Com apenas 50 observações para estimar 10 parâmetros, o modelo M_1 apresenta instabilidade e um coeficiente inconsistente: `mem.used` assume sinal positivo (+0,0263). Isso sugeriria erroneamente que o maior uso de memória aumenta o *frame rate*, o que contraria a lógica do sistema e reflete um claro *overfitting* ao ruído amostral devido à escassez de dados. A partir do modelo M_2 , com 500 amostras, os sinais dos coeficientes se estabilizam em valores negativos ou próximos de zero, e o intercepto converge para a faixa entre 49 e 52. Esse comportamento se mantém essencialmente constante nos modelos seguintes, de 1000 a 2520 amostras.

4.3.2 Tempo de Treinamento e NMAE

A Tabela 6 sintetiza o tempo de treinamento (em milissegundos) e o NMAE no conjunto de teste para cada um dos cinco modelos.

Tabela 6: NMAE e tempo de treinamento dos modelos lineares.

Modelo	n (treino)	Tempo (ms)	NMAE (%)
M_1	50	0.7732	11.5182
M_2	500	0.5827	10.5750
M_3	1000	0.6847	10.4925
M_4	1500	0.7412	10.3776
M_5	2520	0.8030	10.3877

Todos os cinco modelos avaliados ficaram abaixo do limite de 15% para o NMAE, exibindo uma dinâmica clara de convergência. A redução de erro mais expressiva ocorre entre 50 e 500 observações, intervalo em que o NMAE cai de 11,52% para 10,58%, coincidindo com o período em que os coeficientes ganham estabilidade. A partir de 500 amostras, os ganhos tornam-se marginais, com uma melhoria de apenas 0,19 ponto percentual até atingir o modelo completo com 2520 observações, onde o erro estabiliza em torno de 10,4%. Esse comportamento indica que a limitação principal não reside no volume de dados, mas no viés da própria abordagem linear. A bimodalidade do *frame rate* introduz uma não linearidade estrutural que uma regressão linear é incapaz de capturar, independentemente da quantidade de dados oferecida no treinamento.

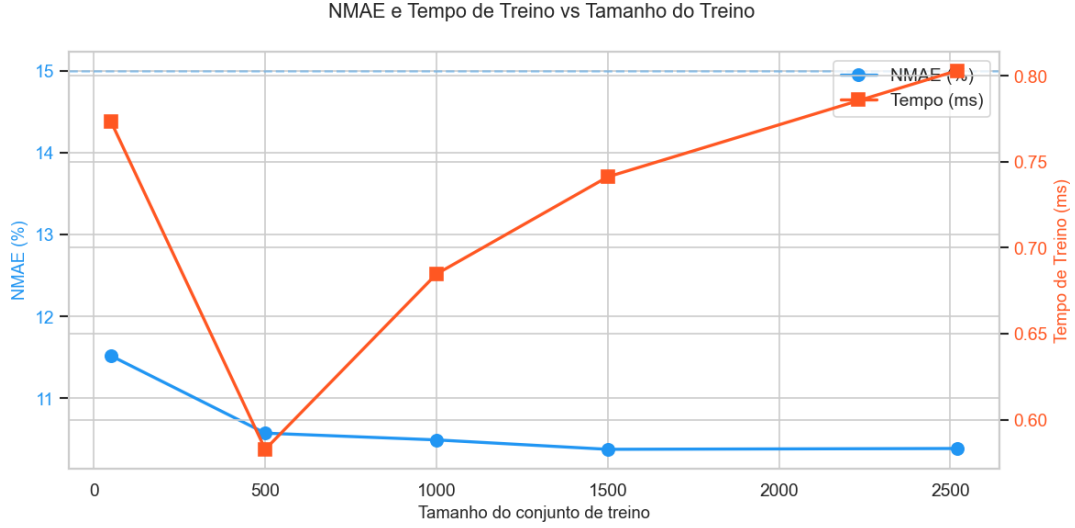


Figura 6: NMAE e tempo de treinamento em função do tamanho do conjunto de treino.

Sob a perspectiva computacional, a solução analítica dos Mínimos Quadrados Ordinários possui complexidade teórica $O(np^2 + p^3)$. Com o número de variáveis preditoras fixado em $p = 9$, seria esperado um crescimento linear do tempo de execução em função de n . Contudo, na prática, todos os tempos mensurados ficaram abaixo de 1 milissegundo e sem uma tendência monotônica clara, ocorrendo situações em que treinos com bases maiores foram concluídos mais rapidamente do que com bases menores. Nessa escala de dados, o tempo real de processamento é da ordem de microssegundos, sendo amplamente dominado por *overheads* fixos do ambiente, como alocação de memória, chamadas às rotinas internas do NumPy e o próprio escalonamento do sistema operacional. Assim, para o volume de dados considerado neste trabalho, o tamanho do conjunto de treinamento tem impacto praticamente nulo sobre o custo computacional, mas influencia diretamente a estabilidade dos parâmetros estimados e a qualidade das predições.

5 Task III - Estimação de Conformidade e Violação do SLA a partir das Estatísticas de Dispositivo

5.1 Transformação do Problema em Classificação Binária

Na Task III, a variável contínua `frame.rate` foi convertida em uma variável binária com base no critério de SLA adotado no projeto. Observações com `frame.rate` maior ou igual a 18 fps foram classificadas como conformes ao SLA, enquanto observações abaixo desse valor foram classificadas como violações. Formalmente, define-se o rótulo binário como:

$$y_{SLA} = \begin{cases} 1, & \text{se } \text{frame.rate} \geq 18, \\ 0, & \text{se } \text{frame.rate} < 18. \end{cases} \quad (3)$$

A definição apresentada anteriormente foi implementada diretamente no código:

```
y_bin = (df['frame.rate'].values >= 18).astype(int)
```

A regressão logística foi utilizada para estimar a probabilidade de conformidade do SLA a partir das estatísticas do servidor, modelando:

$$P(y = 1 | X) = \sigma(\theta_0 + \theta^T \mathbf{x}) = \frac{1}{1 + e^{-(\theta_0 + \theta^T \mathbf{x})}}, \quad (4)$$

em que $\sigma(\cdot)$ representa a função sigmoide. A mesma divisão treino/teste utilizada na Task II foi mantida, com 2520 observações para treinamento (70%) e 1080 para teste (30%). As proporções de

conformidade foram de 52,14% no conjunto de treinamento e 50,46% no conjunto de teste, caracterizando uma distribuição relativamente equilibrada entre as duas classes.

5.2 Impacto do Tamanho do Conjunto de Treinamento no Desempenho do Modelo

Como na Task II, cinco classificadores $C_1, \dots, C_5$ foram treinados sobre subconjuntos de tamanho $n \in \{50, 500, 1000, 1500, 2520\}$, com `LogisticRegression(max_iter=10 000)` do `scikit-learn` para garantir convergência mesmo nos modelos que exigem mais iterações.

5.2.1 Coeficientes dos Classificadores

A Tabela 7 apresenta os coeficientes de cada um dos cinco classificadores.

Tabela 7: Coeficientes dos classificadores C_1 a C_5 , por tamanho do conjunto de treino.

Parâmetro	$n = 50$	$n = 500$	$n = 1000$	$n = 1500$	$n = 2520$
Intercepto	-0.0024	0.0010	0.0014	25.4995	23.6270
cpu.idle	0.0016	-0.0795	-0.0164	-0.0790	-0.0976
mem.used	0.6292	-0.0321	-0.0562	-0.1060	-0.1036
proc.creation.rate	-0.2935	-0.0740	-0.0384	-0.0163	-0.0079
ctx.switch.rate	-0.0012	-0.0000	0.0000	-0.0001	-0.0001
file.handles	-0.0448	0.0037	0.0020	-0.0025	-0.0015
interrupt.rate	0.0067	0.0004	0.0006	0.0001	0.0002
load.avg.1min	-0.3877	-0.0864	-0.0896	-0.0550	-0.0582
tcp.sockets	0.5639	-0.0593	-0.0780	-0.0590	-0.0651
pg.free.rate	0.0005	-0.0000	-0.0000	-0.0000	-0.0000

Com apenas 50 observações, o modelo apresenta maior variabilidade nos coeficientes estimados. Em particular, `mem.used` e `tcp.sockets` assumem coeficientes positivos, diferentemente do observado nos demais modelos. À medida que o tamanho do conjunto de treinamento aumenta, os coeficientes tornam-se mais estáveis e passam a indicar uma relação mais consistente entre o aumento da carga do sistema e a redução da probabilidade de conformidade com o SLA. O intercepto também converge para valores semelhantes nos modelos treinados com 1500 e 2520 observações, sugerindo maior estabilidade da fronteira de decisão.

5.2.2 Tempo de Treinamento e Erro de Classificação (ERR)

O desempenho dos classificadores foi avaliado por meio da métrica *Classification Error* (ERR), definida:

$$\text{ERR} = 1 - \frac{TP + TN}{m}, \quad (5)$$

Essa métrica é equivalente a $1 - \text{acurácia}$, sendo considerado satisfatório um classificador com $\text{ERR} < 15\%$, conforme estabelecido no projeto. Como o conjunto de teste contém $m = 1080$ observações, o cálculo do erro de classificação foi realizado com base nesse total. A Tabela 8 reúne os resultados obtidos para cada modelo, incluindo tempo de treinamento, acurácia, ERR e os componentes da matriz de confusão (TP, TN, FP e FN). Neste contexto, TP representa observações corretamente classificadas como conformes ao SLA, enquanto TN representa observações corretamente classificadas como violações.

Tabela 8: Tempo de treinamento, acurácia, ERR e matriz de confusão para diferentes tamanhos do conjunto de treinamento.

n	Tempo (ms)	Acurácia	ERR (%)	TP	TN	FP	FN
50	69.21	0.8463	15.3704	438	476	59	107
500	251.82	0.8704	12.9630	465	475	60	80
1000	212.73	0.8713	12.8704	464	477	58	81
1500	1322.96	0.8861	11.3889	479	478	57	66
2520	1979.10	0.8889	11.1111	486	474	61	59

O classificador treinado com apenas 50 observações foi o único que não atingiu o critério de desempenho estabelecido no projeto, apresentando ERR de 15,37%. A partir de 500 observações, todos os modelos passaram a satisfazer o limite de 15%, com melhoria gradual até o modelo treinado com o conjunto completo, que obteve ERR de 11,11% e acurácia de 88,89%.

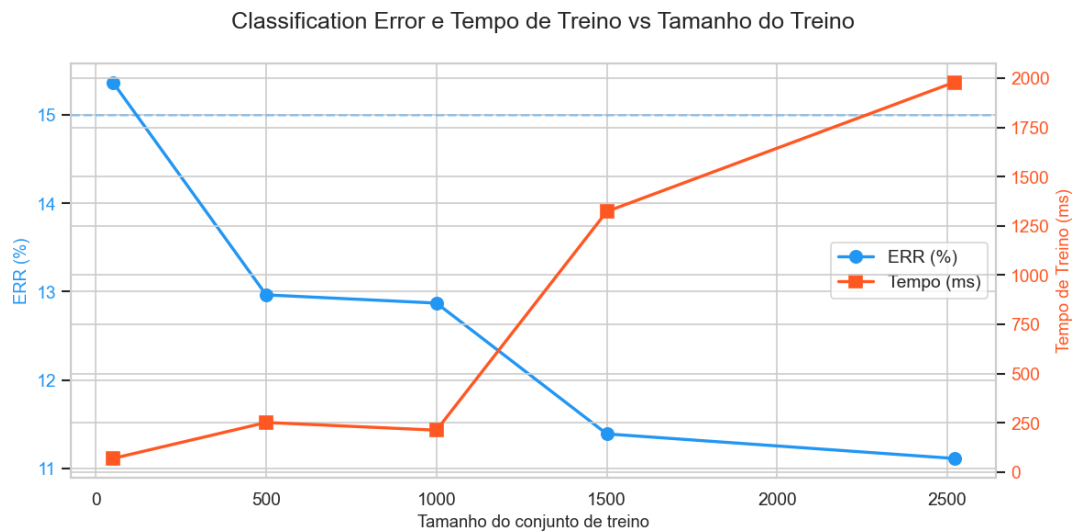


Figura 7: Erro de classificação e tempo de treinamento em função do tamanho do conjunto de treino.

A análise da matriz de confusão mostra que os falsos positivos permanecem relativamente estáveis entre os modelos, variando entre 57 e 61 observações. A principal melhoria ocorre na redução dos falsos negativos, que diminuem de 107 para 59 à medida que o tamanho do conjunto de treinamento aumenta. Isso indica que os ganhos obtidos com mais dados estão associados principalmente à maior capacidade do classificador em identificar corretamente observações conformes ao SLA que anteriormente eram classificadas como violações.

A Figura 7 mostra que o ERR continua diminuindo mesmo após 1000 observações, diferentemente do comportamento observado na regressão linear, cujo erro se estabilizou mais rapidamente. Ao mesmo tempo, o tempo de treinamento cresce de forma significativa para os modelos com 1500 e 2520 observações. Ainda assim, o ganho em desempenho permanece perceptível, indicando que a regressão logística continua se beneficiando do aumento do conjunto de treinamento na definição da fronteira de decisão entre conformidade e violação do SLA.

5.2.3 Número de Iterações

A Tabela 9 apresenta, para cada tamanho do conjunto de treinamento, o erro de classificação (ERR), o tempo de treinamento e o número de iterações necessárias para a convergência do algoritmo L-BFGS utilizado pela regressão logística.

Tabela 9: ERR, tempo e número de iterações da regressão logística.

n	ERR (%)	Tempo (ms)	Iterações
50	15,37	69,21	344
500	12,96	251,82	1249
1000	12,87	212,73	988
1500	11,39	1322,96	5404
2520	11,11	1979,10	6872

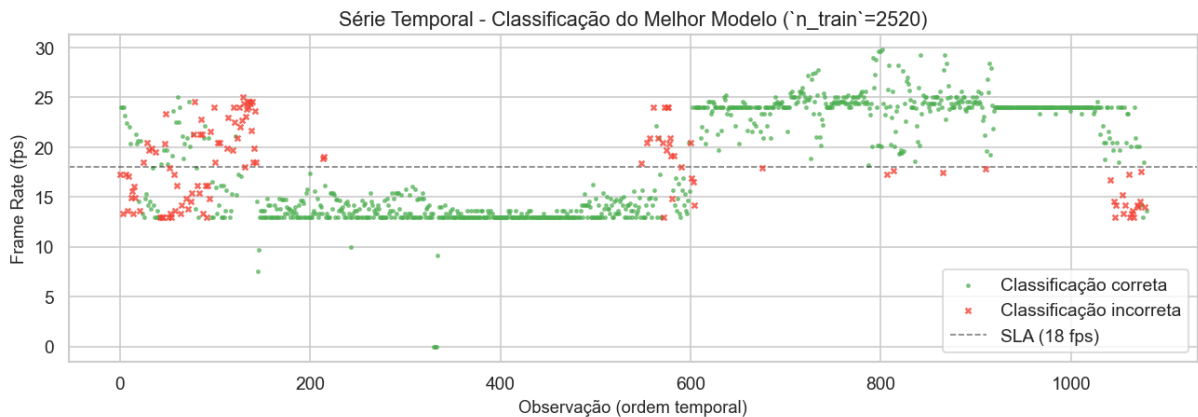
Diferentemente da regressão linear, cujo treinamento foi concluído em menos de 1 ms para todos os experimentos, a regressão logística apresentou tempos significativamente maiores e mais sensíveis ao tamanho do conjunto de treinamento. Como o treinamento é realizado por um processo iterativo de otimização, o tempo total depende não apenas da quantidade de observações, mas também do número de iterações necessárias para atingir a convergência.

Esse comportamento pode ser observado ao comparar os modelos treinados com 500 e 1000 observações. Embora o segundo utilize mais dados, seu tempo de treinamento foi menor (212,73 ms contra 251,82 ms), pois convergiu em menos iterações (988 contra 1249). Já os modelos treinados com 1500 e 2520 observações exigiram 5404 e 6872 iterações, respectivamente, resultando em um aumento expressivo do tempo de execução.

Em termos de desempenho, observa-se uma redução gradual do ERR à medida que o conjunto de treinamento aumenta, passando de 15,37% para 11,11%. Assim, embora conjuntos maiores demandem mais tempo para convergência, eles também produzem classificadores mais precisos. Os resultados indicam que a relação entre tamanho do conjunto de treinamento e tempo de execução não é estritamente linear, uma vez que o comportamento do processo de otimização influencia diretamente o número de iterações necessárias para a convergência.

5.3 Análise do Melhor Classificador

O melhor classificador foi o C_5 , treinado com 2520 observações, apresentando ERR de = 11,11%. A Figura 8 mostra o comportamento desse classificador ao longo da sequência temporal original do conjunto de teste, indicando para cada observação se a decisão tomada foi correta ou incorreta.

**Figura 8:** Série temporal do *frame rate* com acertos e erros do melhor classificador.

A Figura 8 mostra que a maioria dos erros ocorre quando o *frame rate* está próximo do limiar de SLA de 18 fps. Em contrapartida, observações com valores mais distantes desse limite, tanto na região próxima de 13 fps quanto na região próxima de 24 fps, são classificadas corretamente na maior parte dos casos. Esse resultado ajuda a explicar por que o classificador apresenta bom desempenho global, apesar dos erros observados em pontos específicos.

Também é possível notar que os erros não aparecem distribuídos de forma uniforme ao longo da série. Há uma concentração maior nas primeiras 200 observações e novamente por volta das observações 500

a 600. Já entre aproximadamente 300 e 500 observações, bem como entre 800 e 1000, predominam classificações corretas. Visualmente, esses erros parecem ocorrer em trechos nos quais o sistema alterna entre níveis distintos de desempenho, enquanto os acertos se concentram em períodos mais estáveis.

5.4 Relação entre Tamanho do Treino e Acurácia dos Classificadores

A Tabela 10 apresenta os valores de ERR e acurácia obtidos para diferentes tamanhos do conjunto de treinamento.

Tabela 10: ERR e acurácia dos classificadores.

n	ERR (%)	Acurácia (%)
50	15.37	84.63
500	12.96	87.04
1000	12.87	87.13
1500	11.39	88.61
2520	11.11	88.89

O aumento do conjunto de treinamento produz uma redução gradual do erro de classificação. O modelo treinado com apenas 50 observações foi o único que não atingiu o critério estabelecido no projeto, apresentando ERR de 15,37%. Esse resultado é consistente com a maior instabilidade observada nos coeficientes desse classificador. A partir de 500 observações, todos os modelos passaram a apresentar ERR inferior a 15%, com melhoria contínua até o modelo treinado com o conjunto completo, que alcançou ERR de 11,11% e acurácia de 88,89

Diferentemente do observado na Task II, em que o NMAE da regressão linear se estabilizou rapidamente, a regressão logística continuou apresentando ganhos de desempenho à medida que novas observações foram incorporadas ao treinamento. Ainda assim, a redução do erro torna-se progressivamente menor para conjuntos maiores, sugerindo convergência em torno de 11%. Esse resultado indica que o aumento da quantidade de dados melhora a capacidade de classificação do modelo, mas não elimina completamente os erros associados às observações próximas ao limiar de SLA, que representam os casos mais difíceis de distinguir.

5.5 Relação entre Tamanho do Treino e Tempo de Treinamento

A Tabela 9 apresenta o ERR, o tempo de treinamento e o número de iterações necessárias para a convergência da regressão logística em cada tamanho de conjunto de treinamento.

Diferentemente da regressão linear analisada na Task II, cujos modelos foram treinados em menos de 1 ms, a regressão logística apresentou tempos significativamente maiores, variando de 69 ms a aproximadamente 2 s. Essa diferença é consequência do processo iterativo de otimização utilizado para estimar os parâmetros do modelo, em contraste com a solução analítica empregada pela regressão linear.

Observa-se que o tempo de treinamento não depende apenas da quantidade de observações, mas também do número de iterações necessárias para a convergência. Isso explica, por exemplo, por que o modelo treinado com 1000 observações foi mais rápido que o modelo treinado com 500 observações: embora possuísse mais dados, convergiu em menos iterações (988 contra 1249). Por outro lado, os modelos treinados com 1500 e 2520 observações exigiram 5404 e 6872 iterações, respectivamente, resultando em um aumento expressivo do tempo de execução.

Durante os experimentos, o parâmetro `max.iter` precisou ser aumentado para permitir a convergência dos modelos maiores, uma vez que os valores padrão não seriam suficientes para completar o processo de otimização. Apesar do crescimento observado, os tempos obtidos permanecem compatíveis com cenários de análise exploratória e monitoramento offline, especialmente quando comparados ao ganho

de desempenho alcançado pelos classificadores treinados com conjuntos maiores.

6 Conclusão

Este trabalho investigou a utilização de estatísticas de sistema operacional para estimar a qualidade de serviço de uma aplicação de vídeo sob demanda e sua conformidade com um SLA baseado em *frame rate*. Os resultados mostraram que informações coletadas exclusivamente no lado do servidor são capazes de fornecer estimativas úteis tanto para a predição do *frame rate* quanto para a detecção de violações de SLA.

Na Task I, a análise exploratória revelou características importantes do ambiente monitorado, destacando a elevada utilização dos recursos do servidor e a distribuição bimodal da taxa de quadros. Essa característica mostrou-se particularmente relevante para a interpretação dos resultados obtidos nas etapas posteriores.

Na Task II, a regressão linear atingiu NMAE de 10,39%, valor inferior ao limite de 15% estabelecido no projeto. O modelo foi capaz de capturar adequadamente a tendência geral do *frame rate*, embora os maiores erros tenham se concentrado nas regiões de transição entre os dois regimes de operação observados nos dados. Além disso, verificou-se que o erro estabiliza rapidamente com o aumento do conjunto de treinamento, enquanto o tempo de treinamento permanece praticamente desprezível para a escala de dados considerada.

Na Task III, a regressão logística obteve ERR de 11,11%, também satisfazendo o critério de desempenho definido no enunciado. Assim como observado na regressão linear, os erros de classificação concentraram-se principalmente nas proximidades do limiar de SLA, região em que a distinção entre conformidade e violação se torna mais difícil. O aumento do conjunto de treinamento contribuiu para a redução do erro e para a estabilização dos coeficientes do modelo, embora com um custo computacional significativamente superior ao observado na regressão linear.

De forma geral, os resultados indicam que modelos lineares e logísticos constituem soluções simples, interpretáveis e eficazes para a estimação de métricas de serviço e monitoramento de conformidade com SLA. Entretanto, a presença de dois regimes distintos de operação e a concentração dos erros nas regiões de transição sugerem que modelos não lineares podem representar uma alternativa promissora para trabalhos futuros, com potencial para capturar relações mais complexas entre as estatísticas do dispositivo e a qualidade percebida do serviço.

7 Referências

1. G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning*, vol. 112. Springer, 2013.
2. R. Yanggratoke, J. Ahmed, J. Ardelius, C. Flinta, A. Johnsson, D. Gillblad, and R. Stadler. “Predicting real-time service-level metrics from device statistics.” Tech. rep., KTH Royal Institute of Technology, 2014. Disponível em: <http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-152637>.
3. R. Yanggratoke, J. Ahmed, J. Ardelius, C. Flinta, A. Johnsson, D. Gillblad, and R. Stadler. “Linux kernel statistics from a video server and service metrics from a video client.” MLData.org Machine Learning Data Set Repository, 2014. Disponível em: <http://mldata.org/repository/data/viewslug/realml-im2015-vod-traces>.
4. Pasquini, R. “Estimating Conformance to Service Level Agreements (SLAs).” Enunciado do Trabalho Final, PGC308A – Tópicos Especiais em Sistemas de Computação 2: Internet do Futuro, Universidade Federal de Uberlândia, 2026.
5. Pedregosa, F. et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
6. Silverman, B. W. *Density Estimation for Statistics and Data Analysis*. Chapman & Hall, 1986.